

Experiment # 05

Implementation of Serial and Parallel Execution using Threads

Objective:

Objective of this lab is to get the knowledge about Serial and Parallel Execution and check time efficiency.

Software Tools:

- Ubuntu

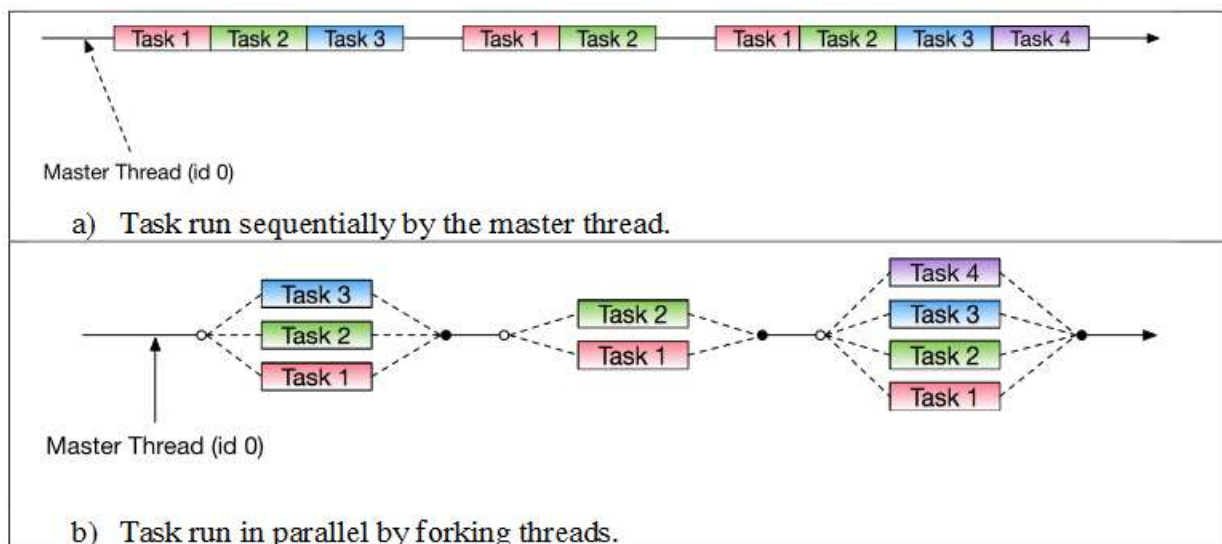
Theory:

What is serial processing

A processing in which one task is completed at a time and all the tasks are run by the processor in a sequence. In real time example; people standing in a queue and waiting for a railway ticket. Similarly, processor gets lists of tasks and each task is completed at a time and all other tasks wait till the first one completes. This type of processing is known as **serial/sequential processing**.

What is parallel processing

It is a type of processing in which multiple tasks are completed at a time by different processors. Note that; in parallel processing there is more than one processor involved. In real time example, there are multiple queues of people standing to get railway tickets. Similarly, in the operating system, there are multiple queues of tasks and multiple tasks are completed by different processors at a time.



What is Threads:

A **thread** is a single sequential flow of control within a program. The real excitement surrounding **threads** is not about a single sequential **thread**. Rather, it's about the use of multiple **threads** running at the same time and performing different tasks in a single program. The implementation of threads and processes differs between operating systems, but in most cases a thread is a component of a process. Multiple threads can exist within one process, executing concurrently and sharing resources such as memory, while different processes do not share these resources. In particular, the threads of a process share its executable code and the values of its dynamically allocated variables and non-thread-local global variables at any given time.

time command:

Time command in Linux is used to execute a command and prints a summary of real-time, user CPU time and system CPU time spent by executing a command when it terminates. '*real*' time is the time elapsed wall clock time taken by a command to get executed, while '*user*' and '*sys*' time are the number of CPU seconds that command uses in user and kernel mode respectively.

Syntax:

time [option] [COMMAND]

Serial Execution:

```
#include<iostream>
#include<thread>
const size_t N = 1e8; double *a, *b, *sum, *diff;
using namespace std;

void init()
{
    for(size_t i=0; i<N; i++)
    {
        a[i] = i*2.1;
        b[i] = i + 3.24;
    }
}

void add()
{
    for(size_t i=0; i<N; i++)
    {
        sum[i] = a[i] + b[i];
    }
}

void sub()
{
    for(size_t i=0; i<N; i++)
    {
        diff[i] = a[i] - b[i];
    }
}
```

```

int main()
{
    cout << "Thread test" << endl;

    // array allocation
    a = new double[N]; b = new double[N];
    sum = new double[N]; diff = new double[N];

    // serial execution
    init();
    add();
    sub();

    //parallel execution
    init();
    //thread t1(add); thread t2(sub); t1.join();
    //t2.join();

    return 0;
}

```

Serial Execution Time output:

```

student@student-Vostro-3671:~/Test$ subl serial.cpp
student@student-Vostro-3671:~/Test$ g++ serial.cpp -o serial -pthread
student@student-Vostro-3671:~/Test$ time ./serial
Thread test
:
real    1m8.513s
user    0m2.166s
sys     0m4.254s

```

Parallel Execution:

```

#include<iostream>
#include<thread>
const size_t N = 1e8; double *a, *b, *sum, *diff;
using namespace std;

void init()
{
    for(size_t i=0; i<N; i++)
    {
        a[i] = i*2.1;
        b[i] = i + 3.24;
    }
}

void add()
{
    for(size_t i=0; i<N; i++)
    {
        sum[i] = a[i] + b[i];
    }
}

void sub()
{
    for(size_t i=0; i<N; i++)
    {
        diff[i] = a[i] - b[i];
    }
}

```

```

int main()
{
    cout << "Thread test" << endl;

    // array allocation
    a = new double[N]; b = new double[N];
    sum = new double[N]; diff = new double[N];

    // serial execution
    // init();
    // add();
    // sub();

    //parallel execution init();
    thread t1(add); thread t2(sub); t1.join();
    t2.join();

    return 0;
}

```

Parallel Execution Time output:

```

student@student-Vostro-3671:~/Test$ gedit parallel.cpp
student@student-Vostro-3671:~/Test$ g++ parallel.cpp -o op -pthread
student@student-Vostro-3671:~/Test$ time ./op
Thread test

real    0m1.909s
user    0m0.807s
sys     0m1.434s

```

In above example code, we calculate the total time of sequential execution and same code will run again but using threads and observed the difference in sequential and parallel execution.

Lab Tasks:

1. Run the above example code and observe the time to execute serially and parallel.
2. Write a C++ program using functions to find:
 - Factorial of the given number.
 - Total number of prime numbers in the given size
 - Cube of each number and add resultant number.
 - For parallel execution, each thread executes one function.
 - Executing same programs using **Timer Library**. And compare results by executing sequential & parallelism.
 - **Draw** table & graph to visualize the speedup.